

PRÁCTICO N° 4

BASE DE DATOS II

ALUMNO: ABACA, JONATAN

PROFESOR: BARRIONUEVO, CESAR

AÑO: 2026

TRANSACTIONS

ESQUEMA DE TABLAS

```
CREATE TABLE clientes (  
  id_cliente INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE,  
  saldo DECIMAL(12,2) DEFAULT 0.00,  
  activo TINYINT(1) DEFAULT 1,  
  fecha_registro DATETIME DEFAULT NOW() );
```

```
CREATE TABLE productos (  
  id_producto INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  precio DECIMAL(10,2) NOT NULL,  
  stock INT DEFAULT 0,  
  fecha_modificacion DATETIME );
```

```
CREATE TABLE ventas (  
  id_venta INT AUTO_INCREMENT PRIMARY KEY,  
  id_cliente INT NOT NULL,  
  id_producto INT NOT NULL,  
  cantidad INT NOT NULL,
```

```
total DECIMAL(12,2),
fecha DATETIME DEFAULT NOW(),
FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente),
FOREIGN KEY (id_producto) REFERENCES productos(id_producto) );

CREATE TABLE auditoria_precios (
id INT AUTO_INCREMENT PRIMARY KEY,
id_producto INT,
precio_anterior DECIMAL(10,2),
precio_nuevo DECIMAL(10,2),
usuario VARCHAR(100),
fecha_cambio DATETIME DEFAULT NOW() );
```

```
CREATE TABLE log_clientes_eliminados (
id INT AUTO_INCREMENT PRIMARY KEY,
id_cliente INT,
nombre VARCHAR(100),
email VARCHAR(150),
fecha_eliminacion DATETIME DEFAULT NOW() );
```

Se agrega la tabla de cuentas al esquema anterior:

```
CREATE TABLE cuentas (
id INT AUTO_INCREMENT PRIMARY KEY,
id_cliente INT NOT NULL,
tipo ENUM('corriente','ahorro') DEFAULT 'ahorro',
saldo DECIMAL(12,2) DEFAULT 0.00,
FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) );
```

```
CREATE TABLE pedidos (
```

```
id INT AUTO_INCREMENT PRIMARY KEY,  
id_cliente INT,  
total DECIMAL(12,2),  
estado VARCHAR(20) DEFAULT 'pendiente' );
```

Ejercicio 9: Transaccion simple con COMMIT y ROLLBACK

Enunciado: Insertar un nuevo cliente y registrar su primera compra. Si ocurre cualquier error, deshacer todo.

```
START TRANSACTION;  
  
INSERT INTO clientes (nombre, email, saldo)  
VALUES ('Ana Torres', 'ana@email.com', 1000.00);  
  
SET @id = LAST_INSERT_ID(); INSERT INTO ventas (id_cliente,  
id_producto, cantidad, total)  
VALUES (@id, 1, 2, 150.00); COMMIT;  
  
-- Si todo OK  
-- ROLLBACK; -- Si algo falla
```

Ejercicio 10: Transferencia bancaria con validacion

Enunciado: Implementar una transferencia entre dos cuentas. Validar que el saldo sea suficiente.

```
DELIMITER $$  
  
CREATE PROCEDURE transferir(  
    IN p_origen INT,  
    IN p_destino INT,  
    IN p_monto DECIMAL(12,2)  
)  
BEGIN  
    DECLARE v_saldo_origen DECIMAL(12,2);
```

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
BEGIN
```

```
    ROLLBACK;
```

```
    SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Error en la transferencia. Operación revertida.';
```

```
END;
```

```
START TRANSACTION;
```

```
    SELECT saldo INTO v_saldo_origen
```

```
    FROM cuentas
```

```
    WHERE id = p_origen
```

```
    FOR UPDATE;      -- bloquea la fila durante la transacción
```

```
    IF v_saldo_origen < p_monto THEN
```

```
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Saldo insuficiente en cuenta origen.';
```

```
    END IF;
```

```
    UPDATE cuentas SET saldo = saldo - p_monto WHERE id = p_origen;
```

```
    UPDATE cuentas SET saldo = saldo + p_monto WHERE id = p_destino;
```

```
COMMIT;
```

```
END$$
```

```
DELIMITER ;
```

```
-- Prueba:
```

```
CALL transferir(1, 2, 500.00);
```

```
SELECT id, saldo FROM cuentas WHERE id IN (1, 2);
```

Ejercicio 11: Uso de SAVEPOINT en proceso de pedido

Enunciado: Registrar un pedido con multiples pasos. Usar SAVEPOINTS para poder deshacer pasos individuales.

```
START TRANSACTION;
```

```
-- Paso 1: registrar el pedido
```

```
INSERT INTO pedidos (id_cliente, total, estado)
```

```
VALUES (1, 350.00, 'pendiente');
```

```
SET @id_pedido = LAST_INSERT_ID();
```

```
SAVEPOINT sp_pedido;    -- punto de retorno seguro
```

```
-- Paso 2: registrar las ventas del pedido
```

```
INSERT INTO ventas (id_cliente, id_producto, cantidad, total)
```

```
VALUES (1, 2, 2, 200.00);
```

```
SAVEPOINT sp_venta1;
```

```
INSERT INTO ventas (id_cliente, id_producto, cantidad, total)
```

```
VALUES (1, 4, 1, 150.00);
```

```
SAVEPOINT sp_venta2;
```

```
-- Si la segunda venta falla, deshacer solo ese paso:
```

```
-- ROLLBACK TO sp_venta1;
```

-- Si todo el detalle falla, volver al pedido base:

-- ROLLBACK TO sp_pedido;

-- Confirmar el estado

UPDATE pedidos SET estado = 'confirmado' WHERE id = @id_pedido;

COMMIT;

Ejercicio 12: Autocommit OFF - Insercion de lote de productos

Enunciado: Insertar un lote de 5 productos en una sola transaccion usando autocommit deshabilitado.

SET autocommit = 0; -- equivale a abrir una transacción implícita

INSERT INTO productos (nombre, precio, stock) VALUES ('Producto A', 100.00, 50);

INSERT INTO productos (nombre, precio, stock) VALUES ('Producto B', 200.00, 30);

INSERT INTO productos (nombre, precio, stock) VALUES ('Producto C', 150.00, 20);

INSERT INTO productos (nombre, precio, stock) VALUES ('Producto D', 80.00, 100);

INSERT INTO productos (nombre, precio, stock) VALUES ('Producto E', 60.00, 200);

-- Si el lote está OK:

COMMIT;

-- Si algún producto tiene datos incorrectos:

-- ROLLBACK;

SET autocommit = 1; -- restaurar siempre al terminar

Ejercicio 13: READ COMMITTED - Demostrar Non-Repeatable Read

Enunciado: Abrir dos sesiones. En la Sesión A, leer el saldo dos veces. Entre lecturas, la Sesión B modifica el saldo. Observar el comportamiento.

TEMA NO VISTO!!

```
-- SESIÓN A: configurar nivel de aislamiento SET SESSION TRANSACTION ISOLATION  
LEVEL READ COMMITTED; START TRANSACTION;
```

```
SELECT saldo FROM cuentas WHERE id = 1;
```

```
-- Resultado: 1000.00
```

```
-- (En este momento, Sesión B ejecuta su UPDATE y hace COMMIT)
```

```
SELECT saldo FROM cuentas WHERE id = 1;
```

```
-- Resultado: 500.00 ← valor diferente en la misma transacción
```

```
COMMIT;
```

```
-- SESIÓN B (ejecutar entre las dos lecturas de A): START TRANSACTION; UPDATE  
cuentas SET saldo = 500.00 WHERE id = 1; COMMIT;
```

Ejercicio 14: FOR UPDATE - Reserva de asientos concurrente

Enunciado: Simular un sistema de reserva donde dos usuarios intentan reservar el ultimo asiento disponible simultaneamente.

```
-- Tabla de contexto (agregar al esquema): -- CREATE TABLE asientos (id INT PK,  
disponible TINYINT(1));
```

```
-- SESIÓN A (llega primero): START TRANSACTION;
```

```
SELECT id FROM asientos
```

```
WHERE disponible = 1
```

```
LIMIT 1
```

```
FOR UPDATE; -- bloquea la fila seleccionada
```

```
-- Resultado: id = 7
```

```
-- Sesión B intenta el mismo SELECT FOR UPDATE → queda en espera
```

```
UPDATE asientos SET disponible = 0 WHERE id = 7;
```

```
COMMIT; -- Recién aquí Sesión B se desbloquea y encuentra 0 filas disponibles
```

```

-- SESIÓN B (llega mientras A tiene el lock): START TRANSACTION;

SELECT id FROM asientos
WHERE disponible = 1
LIMIT 1
FOR UPDATE;
-- Espera hasta que A haga COMMIT, luego no encuentra filas → no
reserva

-- Si no hay filas, manejar en aplicación:
-- SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No hay asientos
disponibles.';

ROLLBACK;

```

Ejercicio 15: DECLARE HANDLER - Captura de errores con ROLLBACK

Enunciado: Crear un procedimiento que intente insertar en una tabla con constraint de clave duplicada, capturando el error con HANDLER.

TEMA NO VISTO!!

DELIMITER \$\$

CREATE PROCEDURE insertar_cliente_seguro(

IN p_nombre VARCHAR(100),

IN p_email VARCHAR(150)

)

BEGIN

DECLARE v_error TINYINT DEFAULT 0;

DECLARE CONTINUE HANDLER FOR SQLEXCEPTION

BEGIN

SET v_error = 1;

END;


```
START TRANSACTION;
```

```
INSERT INTO clientes (nombre, email, saldo)
```

```
VALUES (p_nombre, p_email, 0.00);
```

```
IF v_error = 1 THEN
```

```
    ROLLBACK;
```

```
    SELECT 'Error: el email ya existe o datos inválidos.' AS resultado;
```

```
ELSE
```

```
    COMMIT;
```

```
    SELECT 'Cliente insertado correctamente.' AS resultado;
```

```
END IF;
```

```
END$$
```

```
DELIMITER ;
```

```
-- Prueba normal:
```

```
CALL insertar_cliente_seguro('Luis Gómez', 'luis@email.com');
```

```
-- Prueba con email duplicado (debe capturar el error):
```

```
CALL insertar_cliente_seguro('Luis Copia', 'luis@email.com');
```

Ejercicio 16: Proceso de compra completo - multiples tablas

Enunciado: Implementar el proceso completo de una compra: verificar stock, registrar venta, actualizar stock y debitar saldo del cliente.

```
DELIMITER $$
```

```
CREATE PROCEDURE procesar_compra(
```

```
    IN p_id_cliente INT,
```

```

    IN p_id_producto INT,
    IN p_cantidad INT
)
BEGIN
    DECLARE v_precio DECIMAL(10,2);
    DECLARE v_stock INT;
    DECLARE v_saldo DECIMAL(12,2);
    DECLARE v_total DECIMAL(12,2);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Error en el proceso de compra. Todo fue revertido.';
    END;

    START TRANSACTION;

    -- 1. Leer datos con locks
    SELECT precio, stock
    INTO v_precio, v_stock
    FROM productos
    WHERE id_producto = p_id_producto
    FOR UPDATE;

    SELECT saldo
    INTO v_saldo
    FROM clientes

```

WHERE id_cliente = p_id_cliente

FOR UPDATE;

-- 2. Calcular total

SET v_total = p_cantidad * v_precio;

-- 3. Validar stock

IF v_stock < p_cantidad THEN

 SIGNAL SQLSTATE '45000'

 SET MESSAGE_TEXT = 'Stock insuficiente.';

END IF;

-- 4. Validar saldo

IF v_saldo < v_total THEN

 SIGNAL SQLSTATE '45000'

 SET MESSAGE_TEXT = 'Saldo insuficiente del cliente.';

END IF;

-- 5. Registrar venta

INSERT INTO ventas (id_cliente, id_producto, cantidad, total)

VALUES (p_id_cliente, p_id_producto, p_cantidad, v_total);

-- 6. Descontar stock

UPDATE productos

SET stock = stock - p_cantidad,

 fecha_modificacion = NOW()

WHERE id_producto = p_id_producto;

```
-- 7. Debitar saldo

UPDATE clientes

SET saldo = saldo - v_total

WHERE id_cliente = p_id_cliente;

COMMIT;

SELECT CONCAT('Compra OK. Total: $', v_total) AS resultado;

END$$

DELIMITER ;

-- Prueba:

CALL procesar_compra(1, 2, 3);
```